

Università degli Studi di Milano Bicocca
Scuola di Scienze
Dipartimento di Informatica, Sistemistica e Comunicazione

Metodi del Calcolo Scientifico

Progetto 1

Mini libreria di solutori iterativi per sistemi lineari
simmetrici e definiti positivi

Relazione

Matias Bonoli

Anno Accademico 2025–2026

Indice

1	Introduzione	2
2	Linguaggio e librerie	2
3	I quattro metodi	2
3.1	Criterio di arresto	3
3.2	Schema comune: lo splitting	3
3.3	Metodo di Jacobi	3
3.4	Metodo di Gauss–Seidel	3
3.5	Metodo del Gradiente	4
3.6	Metodo del Gradiente coniugato	4
3.7	Architettura della libreria	4
4	Risultati ed esperimenti	5
4.1	Le matrici di test	5
4.2	Tabelle complete dei risultati	5
4.3	Numero di iterazioni	7
4.4	Errore relativo	9
4.5	Tempo di calcolo	11
4.6	Storia di convergenza	13
4.7	Iterazioni e numero di condizionamento	14
4.8	Costo medio per iterazione	14
4.9	Amplificazione errore/residuo	15
4.10	Riepilogo	16
5	Conclusioni	17
6	Documentazione	18
6.1	Modulo <code>solvers.py</code>	18
6.2	Modulo <code>run_assignment.py</code>	19
6.3	Modulo <code>plots.py</code>	19
6.4	Modulo <code>analyze_cond.py</code>	19
6.5	Modulo <code>test_assignment.py</code>	19
6.6	Esecuzione	19
7	Codice	20
7.1	<code>solvers.py</code>	20
7.2	<code>run_assignment.py</code>	24
7.3	<code>analyze_cond.py</code>	27
7.4	<code>plots.py</code>	28

1 Introduzione

Lo scopo del progetto è scrivere una piccola libreria che risolve sistemi lineari

$$A\mathbf{x} = \mathbf{b}$$

con quattro metodi iterativi: Jacobi, Gauss–Seidel, Gradiente e Gradiente coniugato. La libreria si limita al caso in cui la matrice A è simmetrica e definita positiva (SPD). I quattro metodi sono scritti da zero: la libreria di base serve solo per la struttura dati della matrice e per le operazioni elementari (prodotto matrice–vettore, somma di vettori, prodotto scalare, norma), mentre la parte che calcola la soluzione del sistema è interamente nostra.

Il lavoro segue la procedura standard di validazione. Per ogni matrice di test si fissa la soluzione esatta $\mathbf{x} = [1, 1, \dots, 1]$, si costruisce il termine noto $\mathbf{b} = A\mathbf{x}$, si risolve il sistema con i quattro metodi e si misura, per ciascuno, l’errore relativo rispetto alla soluzione esatta, il numero di iterazioni e il tempo di calcolo. La prova è ripetuta per le quattro tolleranze richieste, $\text{tol} = 10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}$, sempre sullo stesso calcolatore.

La relazione è organizzata così. La Sezione 2 elenca il linguaggio e le librerie usate. La Sezione 3 descrive i quattro metodi, prima dal punto di vista teorico e poi da quello implementativo, e spiega l’architettura della libreria. La Sezione 4 raccoglie i risultati sperimentali in tabelle e grafici, con un commento sotto ognuno. La Sezione 5 riassume le conclusioni. Le Sezioni 6 e 7 contengono la documentazione d’uso e il codice sorgente completo.

2 Linguaggio e librerie

Il progetto è scritto in **Python 3.12**. Tutti i pacchetti usati sono open source:

- **numpy**: array e operazioni vettoriali, cioè prodotto scalare, norma euclidea e aggiornamenti elemento per elemento;
- **scipy.sparse**: struttura dati della matrice in formato sparso CSR e prodotto matrice–vettore Av ;
- **scipy.io.mmread**: lettura dei file in formato MatrixMarket **.mtx**;
- **numba**: compilazione JIT della sola *sweep* di Gauss–Seidel, che è sequenziale e quindi non si può vettorizzare; resta comunque codice nostro;
- **matplotlib**: generazione dei grafici;
- **scipy.sparse.linalg.eigsh**: usato solo nello script a margine che stima gli autovalori estremi e quindi $\text{cond}(A)$, per i commenti. Non fa parte dei solutori.

La traccia chiede di non usare i solutori già pronti della libreria di base. La regola che abbiamo seguito è semplice: di **scipy** usiamo solo la lettura del file, il contenitore sparso CSR e il prodotto matrice–vettore. Nessun metodo che risolve un sistema lineare viene dalla libreria. La scelta del formato sparso non è un dettaglio: le matrici di test hanno migliaia di righe e salvarle per intero costerebbe troppa memoria. Il formato CSR memorizza solo le entrate diverse da zero, e il prodotto Av costa quanto il numero di entrate non nulle invece di $O(n^2)$.

3 I quattro metodi

In questa sezione descriviamo i metodi sul piano teorico e su quello implementativo. Conviene partire da ciò che hanno in comune: il criterio di arresto e lo schema generale di iterazione.

3.1 Criterio di arresto

Tutti i metodi partono dal vettore iniziale nullo $\mathbf{x}^{(0)} = \mathbf{0}$ e costruiscono una successione $\mathbf{x}^{(k)}$ che si avvicina alla soluzione. Non potendo usare la soluzione esatta per fermarsi, si usa il residuo scalato. L'iterazione si arresta quando

$$\frac{\|\mathbf{b} - A\mathbf{x}^{(k)}\|}{\|\mathbf{b}\|} < \text{tol}. \quad (1)$$

Il numeratore è il residuo $\mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)}$. Se è piccolo, allora $A\mathbf{x}^{(k)} \approx \mathbf{b}$, cioè $\mathbf{x}^{(k)}$ risolve il sistema con la precisione richiesta. Si divide per $\|\mathbf{b}\|$ per rendere la quantità adimensionale. Oltre a questo controllo c'è un limite sul numero di iterazioni: se si supera `maxIter` il metodo si ferma e segnala di non aver raggiunto la convergenza. Abbiamo fissato `maxIter` = 20000.

3.2 Schema comune: lo splitting

I primi due metodi, Jacobi e Gauss–Seidel, sono metodi stazionari basati su uno *splitting* della matrice. Si scrive

$$A = P - N, \quad (2)$$

e si riscrive il sistema come $P\mathbf{x} = N\mathbf{x} + \mathbf{b}$, da cui la formula iterativa

$$\mathbf{x}^{(k+1)} = P^{-1}N\mathbf{x}^{(k)} + P^{-1}\mathbf{b}. \quad (3)$$

L'idea è scegliere P in modo che il sistema con matrice P sia facile da risolvere. Manipolando la (3) si ottiene la forma che usiamo nel codice,

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + P^{-1}\mathbf{r}^{(k)}, \quad \mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)}. \quad (4)$$

Il metodo si dice stazionario perché le matrici P e N non dipendono dall'iterazione k . La convergenza è garantita quando il raggio spettrale della matrice di iterazione $P^{-1}N$ è minore di 1; per matrici SPD questo vale per entrambi i metodi.

3.3 Metodo di Jacobi

Si prende come P la diagonale di A e come N il resto con segno opposto. La i -esima componente si aggiorna isolando x_i dalla i -esima equazione e usando i valori dell'iterazione precedente:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right).$$

Dato che P è diagonale, la sua inversa è immediata e $P^{-1}\mathbf{r}$ è solo il prodotto componente per componente fra il residuo e i reciproci della diagonale. Nel codice calcoliamo questi reciproci una volta sola, all'inizio, e a ogni passo aggiorniamo `x += inv_diag * residual` con una operazione vettoriale, senza cicli `for` di Python; poi ricalcoliamo il residuo con `residual = b - A @ x`. Una iterazione costa quindi un prodotto matrice-vettore più operazioni di ordine $O(n)$.

3.4 Metodo di Gauss–Seidel

Si prende come P la parte triangolare inferiore di A , diagonale compresa. La differenza rispetto a Jacobi è che, quando si calcola $x_i^{(k+1)}$, si usano subito i valori già aggiornati in questa stessa iterazione per gli indici $j < i$:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right).$$

Non si calcola mai P^{-1} : dato che P è triangolare inferiore, il sistema $P\mathbf{y} = \mathbf{r}$ si risolve per sostituzione in avanti. Questa sostituzione è sequenziale, perché x_i dipende dai valori $x_0, \dots, x_{i-1}$ appena calcolati, e quindi non si può vettorizzare. Per non pagare un ciclo `for` lento in Python l'abbiamo scritta in una funzione compilata con `numba`, `_gauss_seidel_sweep`, che lavora direttamente sugli array CSR (`indptr`, `indices`, `data`); resta comunque codice nostro, perché `numba` traduce il ciclo in linguaggio macchina ma non risolve il sistema al posto nostro. Dopo la sweep ricalcoliamo il residuo per il criterio di arresto. Anche qui una iterazione costa $O(\text{nnz})$, ma l'uso immediato dei valori aggiornati di solito fa convergere in meno iterazioni di Jacobi.

3.5 Metodo del Gradiente

Per matrici SPD risolvere $A\mathbf{x} = \mathbf{b}$ equivale a cercare il minimo della forma quadratica

$$\phi(\mathbf{y}) = \frac{1}{2} \mathbf{y}^t A \mathbf{y} - \mathbf{b}^t \mathbf{y}, \quad \nabla \phi(\mathbf{y}) = A\mathbf{y} - \mathbf{b}.$$

Il gradiente cambiato di segno coincide con il residuo, quindi la direzione di massima discesa è proprio $\mathbf{r}^{(k)}$. Ci si muove lungo questa direzione di un passo α_k scelto in modo da minimizzare ϕ lungo la retta:

$$\alpha_k = \frac{(\mathbf{r}^{(k)})^t \mathbf{r}^{(k)}}{(\mathbf{r}^{(k)})^t A \mathbf{r}^{(k)}}, \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{r}^{(k)}.$$

Il passo cambia a ogni iterazione, quindi il metodo non è stazionario. Nel codice calcoliamo una sola volta per iterazione il prodotto $A\mathbf{r}^{(k)}$ e lo riusiamo sia per α_k sia per aggiornare il residuo senza un secondo prodotto matrice-vettore, $\mathbf{r}^{(k+1)} = \mathbf{r}^{(k)} - \alpha_k A\mathbf{r}^{(k)}$; questa è la stessa quantità che serve al criterio di arresto, quindi il residuo è già disponibile. Il metodo converge per ogni dato iniziale, ma il numero di iterazioni cresce con il numero di condizionamento di A : quando gli autovalori sono molto diversi la discesa procede a zig-zag e servono molti passi.

3.6 Metodo del Gradiente coniugato

Il difetto del gradiente è che può tornare a muoversi lungo una direzione su cui era già ottimale, da cui lo zig-zag. Il gradiente coniugato sceglie le direzioni $\mathbf{d}^{(k)}$ in modo che siano A -coniugate, cioè $(\mathbf{d}^{(i)})^t A \mathbf{d}^{(j)} = 0$ per $i \neq j$; così, una volta minimizzato lungo una direzione, non serve più tornarci. La direzione nuova è il residuo corretto con la direzione precedente:

$$\alpha_k = \frac{(\mathbf{r}^{(k)})^t \mathbf{r}^{(k)}}{(\mathbf{d}^{(k)})^t A \mathbf{d}^{(k)}}, \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{d}^{(k)},$$

$$\beta_k = \frac{(\mathbf{r}^{(k+1)})^t \mathbf{r}^{(k+1)}}{(\mathbf{r}^{(k)})^t \mathbf{r}^{(k)}}, \quad \mathbf{d}^{(k+1)} = \mathbf{r}^{(k+1)} + \beta_k \mathbf{d}^{(k)}.$$

In aritmetica esatta il metodo converge in al più n iterazioni; in pratica il numero di iterazioni cresce circa come $\sqrt{\text{cond}(A)}$, molto meno del gradiente semplice. Come per il gradiente, nel codice si calcola un solo prodotto matrice-vettore per iterazione, $A\mathbf{d}^{(k)}$, riusato per α_k e per aggiornare il residuo; teniamo da parte $\|\mathbf{r}^{(k)}\|^2$ così che β_k e il valore del residuo per il criterio di arresto non costino calcoli aggiuntivi.

3.7 Architettura della libreria

La traccia chiede una libreria strutturata, non una sequenza di funzioni scollegate. I quattro metodi condividono lo stesso scheletro: partono da $\mathbf{x}^{(0)} = \mathbf{0}$, ripetono un passo di aggiornamento, controllano il residuo scalato (1) e si fermano per convergenza o per numero massimo di iterazioni. L'unica cosa che cambia da un metodo all'altro è la regola di aggiornamento e come si calcola il residuo.

Abbiamo quindi scritto lo scheletro una volta sola. La classe base astratta `IterativeSolver` contiene il metodo `solve`, che gestisce il vettore iniziale, il ciclo di arresto, la misura del tempo e l'eventuale storia del residuo. Ogni passo è incapsulato in un oggetto `_Iteration`, una classe astratta con due soli metodi: `residual_norm()`, che dà $\|\mathbf{r}^{(k)}\|$, e `step()`, che porta $\mathbf{x}^{(k)}$ in $\mathbf{x}^{(k+1)}$. Ogni metodo è una sottoclasse che fornisce solo la propria iterazione. Il risultato di una risoluzione è raccolto in un oggetto `SolverResult` con metodo, soluzione, iterazioni, tempo, residuo raggiunto e flag di convergenza. I quattro solutori sono raccolti nella tupla `SOLVERS` nell'ordine della traccia. Prima delle misure di tempo chiamiamo `warmup()`, che esegue i metodi su un problema 3×3 per pagare una volta sola la compilazione JIT di `numba` e non falsare i tempi.

Costo per iterazione. In tutti e quattro i metodi il costo di una iterazione è dominato da un prodotto matrice–vettore, $O(\text{nnz})$, più operazioni vettoriali $O(n)$. La differenza pratica è che Jacobi e Gauss–Seidel ricalcolano il residuo con un prodotto $A\mathbf{x}$ esplicito a fine passo, mentre Gradiente e Gradiente coniugato ottengono il residuo aggiornato per ricorsione, riusando il prodotto già fatto per il passo. Il numero totale di iterazioni dipende invece molto dal metodo e dal condizionamento della matrice, ed è qui che i metodi si distinguono davvero.

4 Risultati ed esperimenti

Tutti i contenuti prodotti dagli script sono raccolti nella cartella `results`: il file `results.csv` con le misure numeriche, il file `cond.csv` con i numeri di condizionamento e la cartella `plots` con i grafici. In questa sezione li riportiamo tutti, ognuno con un commento.

4.1 Le matrici di test

La Tabella 1 riassume le quattro matrici, con dimensione, numero di entrate non nulle, densità e gli autovalori estremi stimati con `analyze_cond.py`. I dati vengono dal file `cond.csv`.

Tabella 1: Matrici di test. La densità è nnz/n^2 . Gli autovalori e il numero di condizionamento $\kappa(A) = \lambda_{\max}/\lambda_{\min}$ sono stimati numericamente.

Matrice	n	nnz	densità	$\lambda_{\min}$	$\lambda_{\max}$	$\kappa(A)$
spa1.mtx	1000	182 434	18.2%	$4.88 \cdot 10^{-1}$	$9.99 \cdot 10^2$	$2.05 \cdot 10^3$
spa2.mtx	3000	1 633 298	18.1%	2.12	$3.00 \cdot 10^3$	$1.41 \cdot 10^3$
vem1.mtx	1681	13 385	0.47%	$1.23 \cdot 10^{-2}$	4.00	$3.25 \cdot 10^2$
vem2.mtx	2601	21 225	0.31%	$7.89 \cdot 10^{-3}$	4.00	$5.07 \cdot 10^2$

Commento. Ci sono due famiglie diverse. Le matrici `spa` sono di media densità, circa il 18% di entrate non nulle, con autovalore massimo grande. Le matrici `vem` sono molto sparse, sotto lo 0.5%, con autovalore massimo limitato a 4 ma autovalore minimo molto piccolo. Il condizionamento più alto è quello di `spa1`. Questa differenza di struttura spiega gran parte di ciò che vedremo: il costo di una iterazione dipende dal numero di entrate non nulle, mentre il numero di iterazioni dipende soprattutto dal condizionamento e dal metodo.

4.2 Tabelle complete dei risultati

Le Tabelle 2–5 riportano, per ogni matrice e ogni tolleranza, il numero di iterazioni, l'errore relativo rispetto alla soluzione esatta, il residuo scalato raggiunto e il tempo. Sono i dati di `results.csv`. In tutte le prove i quattro metodi hanno raggiunto la convergenza: nessuno ha toccato il limite di 20000 iterazioni.

Tabella 2: Risultati su **spa1.mtx** ($n = 1000$, $\kappa \approx 2.0 \cdot 10^3$).

Metodo	tol	Iter.	Errore rel.	Residuo rel.	Tempo [s]
Jacobi	10^{-4}	115	$1.77 \cdot 10^{-3}$	$9.48 \cdot 10^{-5}$	0.013
Jacobi	10^{-6}	181	$1.80 \cdot 10^{-5}$	$9.62 \cdot 10^{-7}$	0.021
Jacobi	10^{-8}	247	$1.82 \cdot 10^{-7}$	$9.76 \cdot 10^{-9}$	0.028
Jacobi	10^{-10}	313	$1.85 \cdot 10^{-9}$	$9.91 \cdot 10^{-11}$	0.035
Gauss-Seidel	10^{-4}	9	$1.82 \cdot 10^{-2}$	$7.79 \cdot 10^{-5}$	0.002
Gauss-Seidel	10^{-6}	17	$1.30 \cdot 10^{-4}$	$5.58 \cdot 10^{-7}$	0.004
Gauss-Seidel	10^{-8}	24	$1.71 \cdot 10^{-6}$	$7.34 \cdot 10^{-9}$	0.006
Gauss-Seidel	10^{-10}	31	$2.25 \cdot 10^{-8}$	$9.65 \cdot 10^{-11}$	0.008
Gradiente	10^{-4}	143	$3.46 \cdot 10^{-2}$	$9.91 \cdot 10^{-5}$	0.016
Gradiente	10^{-6}	3577	$9.68 \cdot 10^{-4}$	$9.98 \cdot 10^{-7}$	0.407
Gradiente	10^{-8}	8233	$9.82 \cdot 10^{-6}$	$9.99 \cdot 10^{-9}$	0.930
Gradiente	10^{-10}	12919	$9.82 \cdot 10^{-8}$	$9.99 \cdot 10^{-11}$	1.463
Grad. coniugato	10^{-4}	49	$2.08 \cdot 10^{-2}$	$9.78 \cdot 10^{-5}$	0.006
Grad. coniugato	10^{-6}	134	$2.55 \cdot 10^{-5}$	$9.74 \cdot 10^{-7}$	0.015
Grad. coniugato	10^{-8}	177	$1.32 \cdot 10^{-7}$	$9.03 \cdot 10^{-9}$	0.020
Grad. coniugato	10^{-10}	200	$1.20 \cdot 10^{-9}$	$7.94 \cdot 10^{-11}$	0.023

Tabella 3: Risultati su **spa2.mtx** ($n = 3000$, $\kappa \approx 1.4 \cdot 10^3$).

Metodo	tol	Iter.	Errore rel.	Residuo rel.	Tempo [s]
Jacobi	10^{-4}	36	$1.77 \cdot 10^{-3}$	$9.54 \cdot 10^{-5}$	0.043
Jacobi	10^{-6}	57	$1.67 \cdot 10^{-5}$	$9.00 \cdot 10^{-7}$	0.067
Jacobi	10^{-8}	78	$1.57 \cdot 10^{-7}$	$8.49 \cdot 10^{-9}$	0.097
Jacobi	10^{-10}	99	$1.48 \cdot 10^{-9}$	$8.01 \cdot 10^{-11}$	0.119
Gauss-Seidel	10^{-4}	5	$2.60 \cdot 10^{-3}$	$4.14 \cdot 10^{-5}$	0.013
Gauss-Seidel	10^{-6}	8	$5.14 \cdot 10^{-5}$	$7.85 \cdot 10^{-7}$	0.020
Gauss-Seidel	10^{-8}	12	$2.79 \cdot 10^{-7}$	$4.27 \cdot 10^{-9}$	0.031
Gauss-Seidel	10^{-10}	15	$5.57 \cdot 10^{-9}$	$8.52 \cdot 10^{-11}$	0.037
Gradiente	10^{-4}	161	$1.81 \cdot 10^{-2}$	$9.90 \cdot 10^{-5}$	0.186
Gradiente	10^{-6}	1949	$6.69 \cdot 10^{-4}$	$9.99 \cdot 10^{-7}$	2.280
Gradiente	10^{-8}	5087	$6.87 \cdot 10^{-6}$	$9.98 \cdot 10^{-9}$	6.028
Gradiente	10^{-10}	8285	$6.94 \cdot 10^{-8}$	$9.99 \cdot 10^{-11}$	9.891
Grad. coniugato	10^{-4}	42	$9.82 \cdot 10^{-3}$	$9.91 \cdot 10^{-5}$	0.049
Grad. coniugato	10^{-6}	122	$1.20 \cdot 10^{-4}$	$9.09 \cdot 10^{-7}$	0.144
Grad. coniugato	10^{-8}	196	$5.59 \cdot 10^{-7}$	$9.60 \cdot 10^{-9}$	0.232
Grad. coniugato	10^{-10}	240	$5.32 \cdot 10^{-9}$	$9.82 \cdot 10^{-11}$	0.291

Tabella 4: Risultati su `vem1.mtx` ($n = 1681$, $\kappa \approx 3.2 \cdot 10^2$).

Metodo	tol	Iter.	Errore rel.	Residuo rel.	Tempo [s]
Jacobi	10^{-4}	1314	$3.54 \cdot 10^{-3}$	$9.99 \cdot 10^{-5}$	0.015
Jacobi	10^{-6}	2433	$3.54 \cdot 10^{-5}$	$9.99 \cdot 10^{-7}$	0.029
Jacobi	10^{-8}	3552	$3.54 \cdot 10^{-7}$	$9.99 \cdot 10^{-9}$	0.040
Jacobi	10^{-10}	4671	$3.54 \cdot 10^{-9}$	$9.99 \cdot 10^{-11}$	0.052
Gauss-Seidel	10^{-4}	659	$3.51 \cdot 10^{-3}$	$9.93 \cdot 10^{-5}$	0.018
Gauss-Seidel	10^{-6}	1218	$3.53 \cdot 10^{-5}$	$9.99 \cdot 10^{-7}$	0.032
Gauss-Seidel	10^{-8}	1778	$3.52 \cdot 10^{-7}$	$9.96 \cdot 10^{-9}$	0.048
Gauss-Seidel	10^{-10}	2338	$3.51 \cdot 10^{-9}$	$9.94 \cdot 10^{-11}$	0.063
Gradiente	10^{-4}	890	$2.70 \cdot 10^{-3}$	$9.93 \cdot 10^{-5}$	0.013
Gradiente	10^{-6}	1612	$2.71 \cdot 10^{-5}$	$9.97 \cdot 10^{-7}$	0.023
Gradiente	10^{-8}	2336	$2.70 \cdot 10^{-7}$	$9.90 \cdot 10^{-9}$	0.033
Gradiente	10^{-10}	3058	$2.71 \cdot 10^{-9}$	$9.97 \cdot 10^{-11}$	0.043
Grad. coniugato	10^{-4}	38	$4.08 \cdot 10^{-5}$	$7.11 \cdot 10^{-5}$	0.0006
Grad. coniugato	10^{-6}	45	$3.73 \cdot 10^{-7}$	$8.90 \cdot 10^{-7}$	0.0007
Grad. coniugato	10^{-8}	53	$2.83 \cdot 10^{-9}$	$7.80 \cdot 10^{-9}$	0.0008
Grad. coniugato	10^{-10}	59	$2.19 \cdot 10^{-11}$	$6.91 \cdot 10^{-11}$	0.0009

Tabella 5: Risultati su `vem2.mtx` ($n = 2601$, $\kappa \approx 5.1 \cdot 10^2$).

Metodo	tol	Iter.	Errore rel.	Residuo rel.	Tempo [s]
Jacobi	10^{-4}	1927	$4.97 \cdot 10^{-3}$	$9.99 \cdot 10^{-5}$	0.029
Jacobi	10^{-6}	3676	$4.97 \cdot 10^{-5}$	$9.99 \cdot 10^{-7}$	0.055
Jacobi	10^{-8}	5425	$4.97 \cdot 10^{-7}$	$9.99 \cdot 10^{-9}$	0.082
Jacobi	10^{-10}	7174	$4.96 \cdot 10^{-9}$	$9.98 \cdot 10^{-11}$	0.110
Gauss-Seidel	10^{-4}	965	$4.95 \cdot 10^{-3}$	$9.98 \cdot 10^{-5}$	0.036
Gauss-Seidel	10^{-6}	1840	$4.94 \cdot 10^{-5}$	$9.96 \cdot 10^{-7}$	0.069
Gauss-Seidel	10^{-8}	2714	$4.96 \cdot 10^{-7}$	$9.99 \cdot 10^{-9}$	0.104
Gauss-Seidel	10^{-10}	3589	$4.95 \cdot 10^{-9}$	$9.97 \cdot 10^{-11}$	0.134
Gradiente	10^{-4}	1308	$3.81 \cdot 10^{-3}$	$9.98 \cdot 10^{-5}$	0.024
Gradiente	10^{-6}	2438	$3.79 \cdot 10^{-5}$	$9.92 \cdot 10^{-7}$	0.045
Gradiente	10^{-8}	3566	$3.81 \cdot 10^{-7}$	$9.97 \cdot 10^{-9}$	0.065
Gradiente	10^{-10}	4696	$3.80 \cdot 10^{-9}$	$9.94 \cdot 10^{-11}$	0.084
Grad. coniugato	10^{-4}	47	$5.73 \cdot 10^{-5}$	$9.19 \cdot 10^{-5}$	0.0009
Grad. coniugato	10^{-6}	56	$4.74 \cdot 10^{-7}$	$8.54 \cdot 10^{-7}$	0.0011
Grad. coniugato	10^{-8}	66	$4.30 \cdot 10^{-9}$	$8.60 \cdot 10^{-9}$	0.0013
Grad. coniugato	10^{-10}	74	$2.25 \cdot 10^{-11}$	$5.90 \cdot 10^{-11}$	0.0013

Commento. Le tabelle mostrano due fatti generali. Primo, il residuo finale è sempre appena sotto la tolleranza richiesta: il criterio di arresto funziona come previsto. Secondo, l'errore relativo scala come la tolleranza. Quando `tol` scende di un fattore 100, anche l'errore scende di circa 100. Questo dice che errore e residuo sono proporzionali, e il fattore di proporzionalità dipende dalla matrice e dal metodo. Sul resto, cioè quanto costano in iterazioni e in tempo, i metodi si comportano in modo molto diverso, e lo analizziamo nei grafici seguenti.

4.3 Numero di iterazioni

Il grafico in Figura 1 mostra il numero di iterazioni in funzione della tolleranza, una riga per ogni matrice.

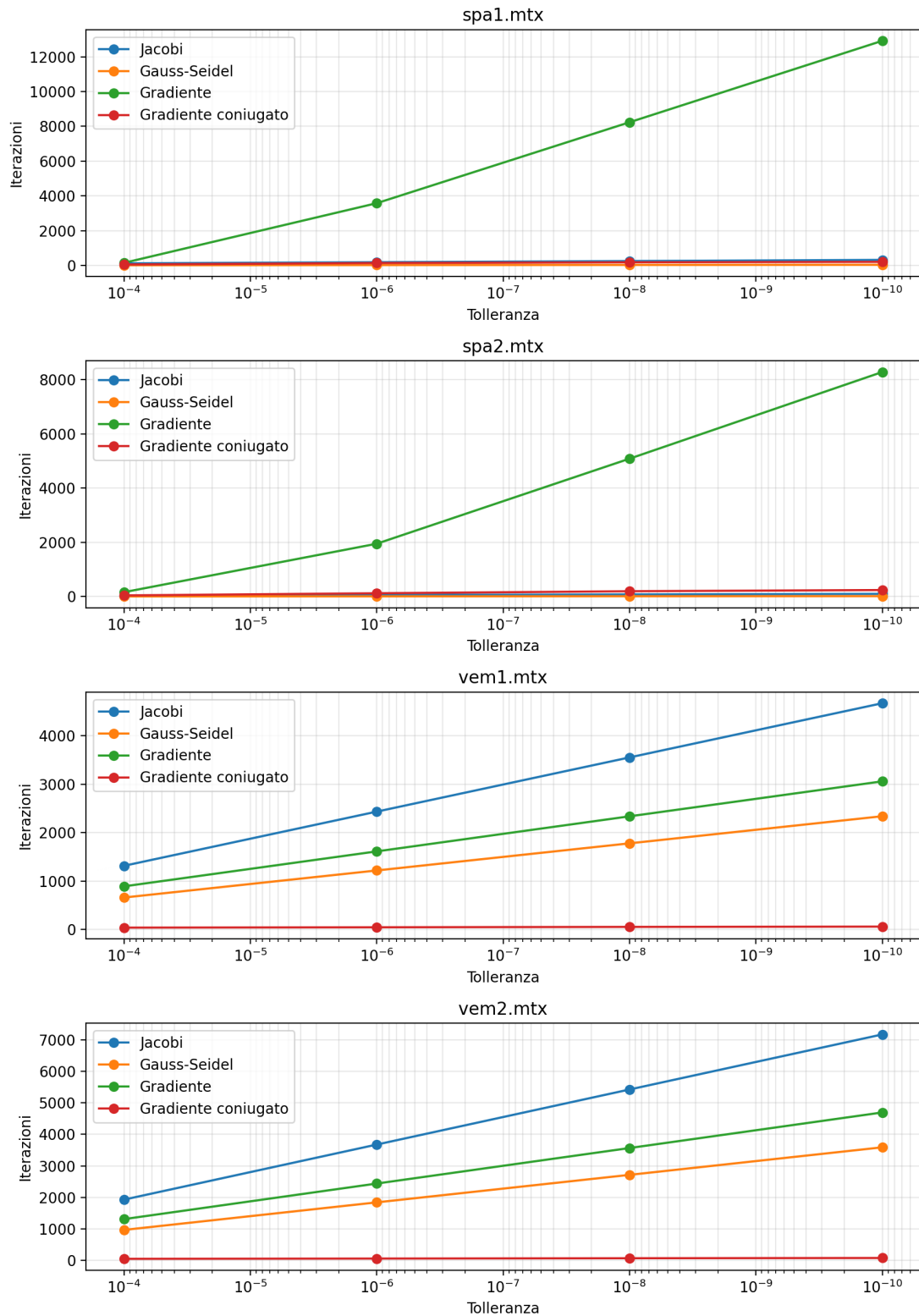


Figura 1: Numero di iterazioni al variare della tolleranza, per ciascun metodo e ciascuna matrice. Asse delle ascisse in scala logaritmica, con la tolleranza che cala da sinistra a destra.

Commento. Il gradiente coniugato richiede sempre pochissime iterazioni, da qualche decina a un paio di centinaia, e la sua linea resta quasi piatta in basso. Il gradiente semplice è il più lento sulle matrici `spa`, dove arriva a quasi 13000 iterazioni su `spa1`, perché il condizionamento alto lo fa procedere a zig-zag. Sulle matrici `vem` il gradiente è invece in linea con Jacobi e Gauss-Seidel.

Gauss–Seidel usa sempre meno iterazioni di Jacobi, in genere circa la metà, perché riusa subito i valori appena aggiornati. Si vede anche che per tutti i metodi il numero di iterazioni cresce in modo regolare al diminuire della tolleranza: passare da 10^{-4} a 10^{-10} aggiunge iterazioni, ma in modo lineare nel numero di cifre richieste.

4.4 Errore relativo

La Figura 2 mostra l'errore relativo rispetto alla soluzione esatta.

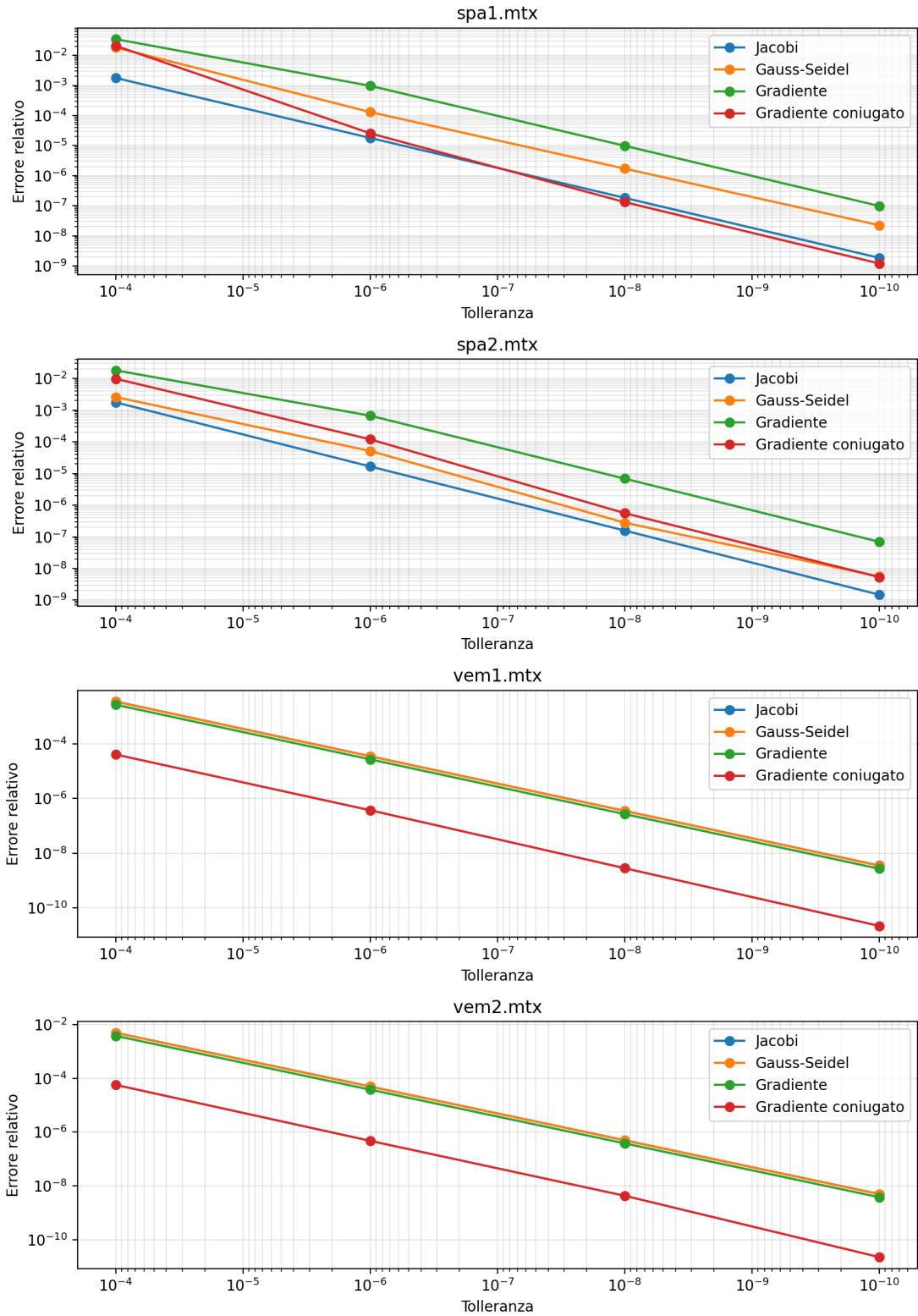


Figura 2: Errore relativo $\|\mathbf{x} - \mathbf{x}^{(k)}\|/\|\mathbf{x}\|$ al variare della tolleranza. Entrambi gli assi in scala logaritmica.

Commento. Le rette scendono in modo regolare: l'errore è proporzionale alla tolleranza, come avevamo notato dalle tabelle. La pendenza è quasi la stessa per tutti, segno che la proporzionalità tra errore e residuo non cambia con la tolleranza. Cambia invece la posizione delle rette. Sulle matrici **vem** il gradiente coniugato sta nettamente più in basso degli altri tre: a parità di

tolleranza raggiunge un errore più piccolo. Sulle **spa** le quattro rette sono più vicine. Vale la pena notare che l'errore non è uguale alla tolleranza ma può essere più grande o più piccolo: il legame tra residuo ed errore passa per il condizionamento della matrice, come riprendiamo nel grafico di amplificazione.

4.5 Tempo di calcolo

La Figura 3 mostra il tempo di calcolo.

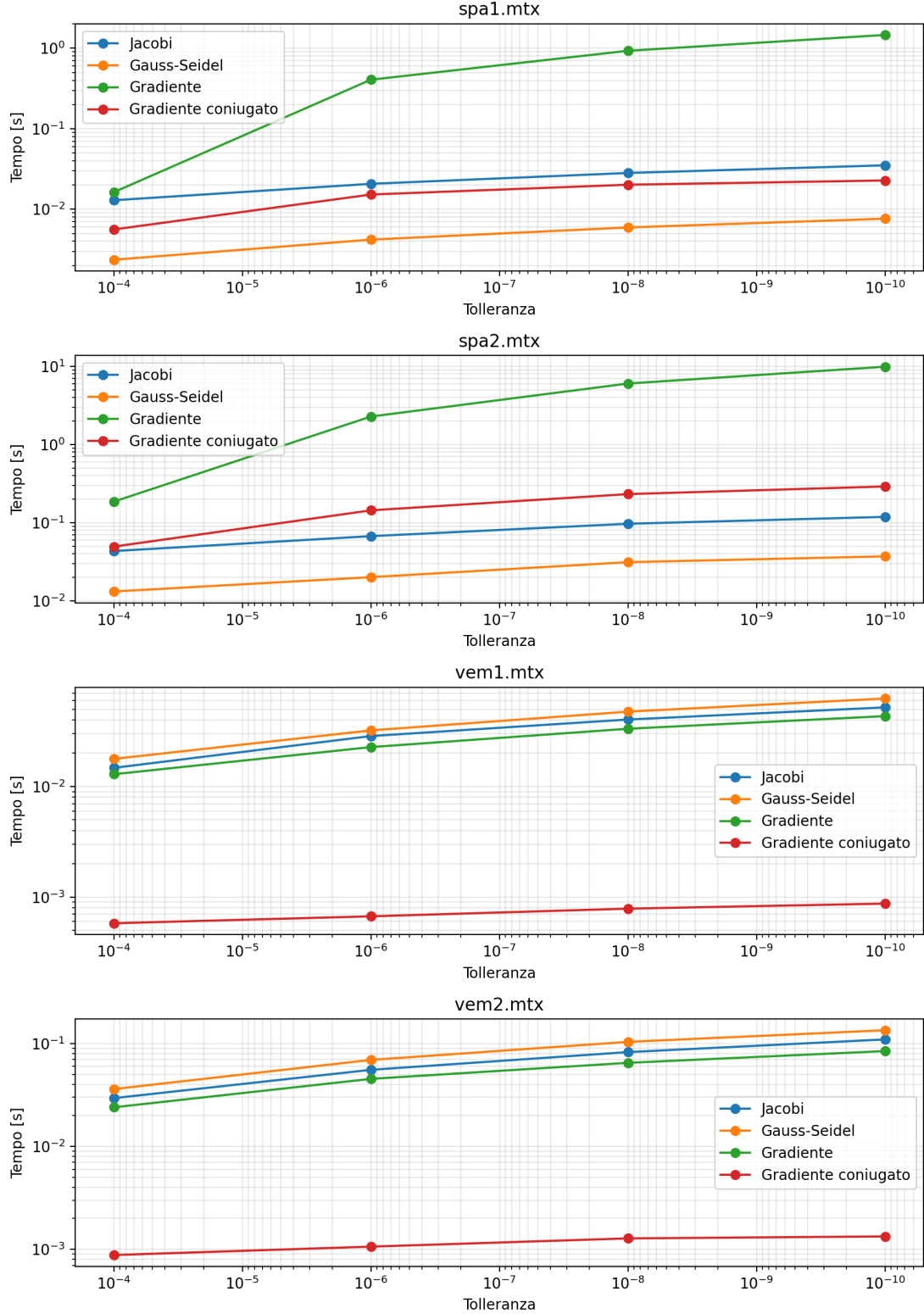


Figura 3: Tempo di calcolo al variare della tolleranza. Asse delle ordinate in scala logaritmica.

Commento. Il tempo segue il numero di iterazioni moltiplicato per il costo di una iterazione. Sulle matrici **spa** il più veloce è Gauss-Seidel, perché converge in pochissime iterazioni; il gradiente coniugato è secondo e il gradiente semplice è di gran lunga il più lento, fino a quasi 10 secondi su **spa2**. Sulle matrici **vem** la situazione si ribalta: il gradiente coniugato è il più veloce con un margine enorme, sotto il millesimo di secondo, mentre gli altri tre stanno vicini tra loro

e molto più in alto. Il motivo è che su **vem** gli altri metodi richiedono migliaia di iterazioni, mentre il gradiente coniugato ne basta qualche decina.

4.6 Storia di convergenza

La Figura 4 mostra come scende il residuo scalato iterazione dopo iterazione, alla tolleranza più stretta 10^{-10} .

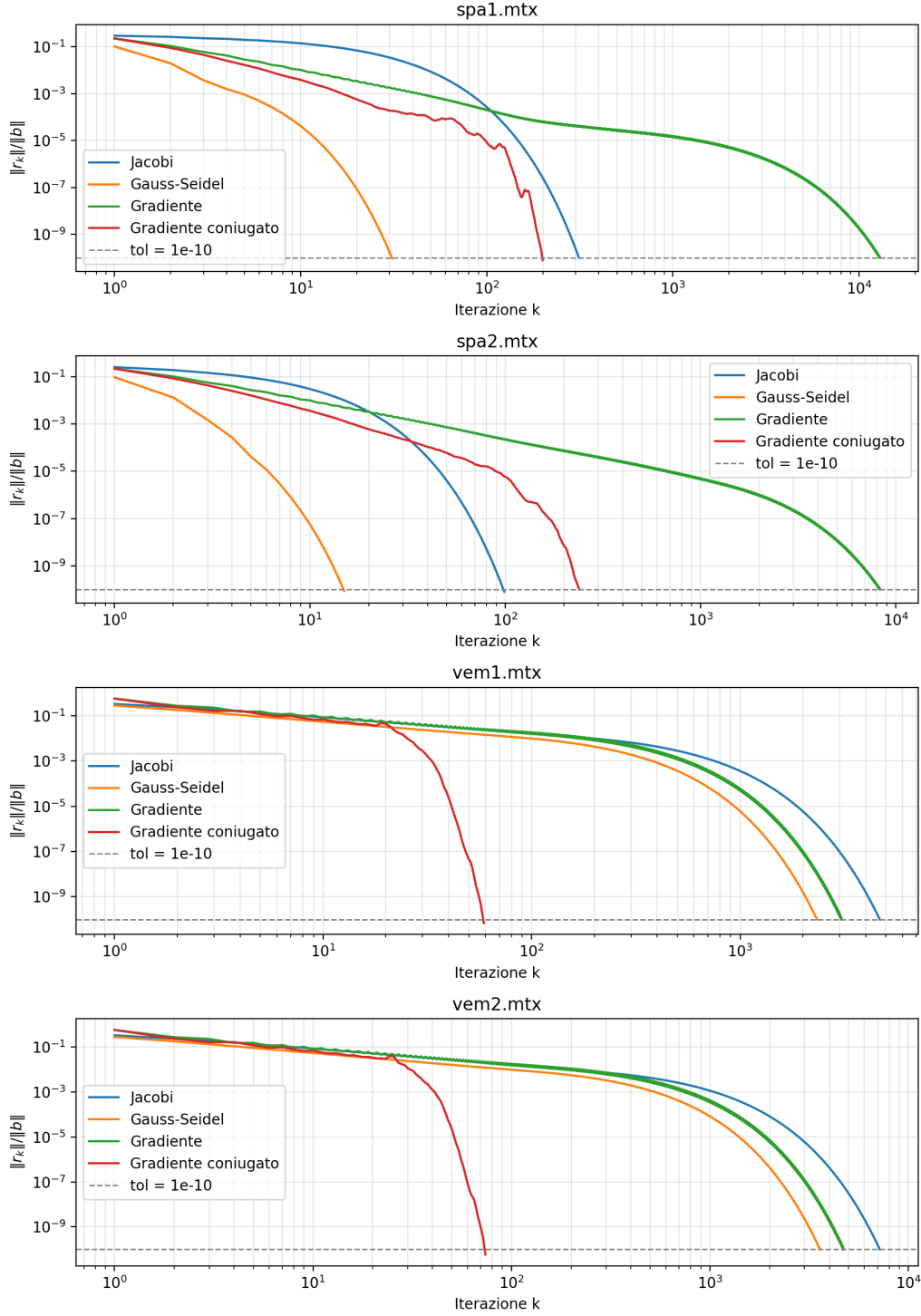


Figura 4: Andamento del residuo scalato $\|r_k\|/\|b\|$ in funzione dell'iterazione k , per $\text{tol} = 10^{-10}$. Entrambi gli assi in scala logaritmica. La riga tratteggiata è la tolleranza.

Commento. Le curve dicono come converge ogni metodo. Sulle matrici **spa** Gauss–Seidel scende per primo, in modo ripido; il gradiente semplice ha una coda lunghissima, segno della convergenza lenta a zig-zag. Il gradiente coniugato ha un andamento irregolare ma scende molto più in fretta del gradiente, fino a un crollo finale. Sulle matrici **vem** il gradiente coniugato si stacca nettamente: dopo poche decine di iterazioni precipita sotto la tolleranza come un muro, mentre Jacobi, Gauss–Seidel e gradiente continuano a scendere piano per migliaia di iterazioni quasi sovrapposti.

4.7 Iterazioni e numero di condizionamento

La Figura 5 mette le iterazioni in funzione del numero di condizionamento, con due curve teoriche di riferimento.

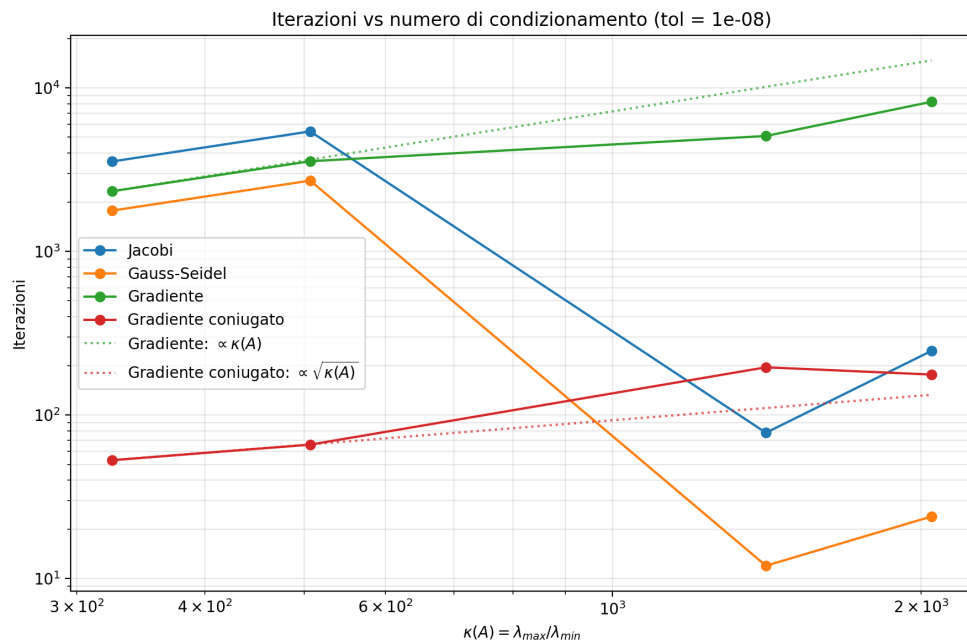


Figura 5: Iterazioni in funzione di $\kappa(A)$ a $\text{tol} = 10^{-8}$. Le linee tratteggiate sono le crescite teoriche attese, $\propto \kappa$ per il gradiente e $\propto \sqrt{\kappa}$ per il gradiente coniugato, ancorate al punto a condizionamento minimo.

Commento. Per il gradiente e il gradiente coniugato l'andamento segue le curve teoriche: il gradiente cresce circa come $\kappa(A)$ e il gradiente coniugato circa come $\sqrt{\kappa(A)}$, molto più piano. Questo conferma la teoria. Le curve di Jacobi e Gauss–Seidel invece non sono monotone in $\kappa(A)$: ad esempio Gauss–Seidel crolla a poche iterazioni sulle matrici **spa**, che pure hanno il condizionamento più alto. Il motivo è che il numero di condizionamento da solo non basta a prevedere le iterazioni di questi due metodi: conta anche la struttura della matrice e la distribuzione degli autovalori. I quattro punti, inoltre, vengono da due famiglie di matrici diverse, quindi il confronto va letto come tendenza, non come legge esatta.

4.8 Costo medio per iterazione

La Figura 6 mostra il tempo medio di una singola iterazione, cioè il tempo totale diviso per il numero di iterazioni.

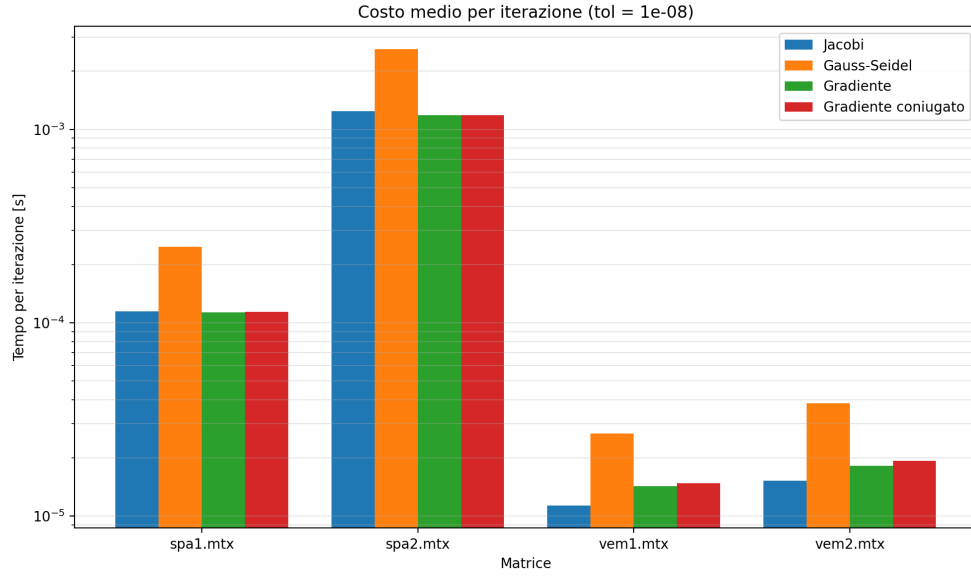


Figura 6: Tempo medio per iterazione a $\text{tol} = 10^{-8}$. Asse delle ordinate in scala logaritmica.

Commento. Questo grafico isola il costo di un passo dal numero di passi. Il costo per iterazione cresce con il numero di entrate non nulle: **spa2**, la matrice più grande e con più entrate, è la più cara per tutti i metodi, mentre le **vem**, molto sparse, sono le più economiche. A parità di matrice i tre metodi che fanno un solo prodotto matrice-vettore, cioè Jacobi, gradiente e gradiente coniugato, hanno costo per iterazione simile. Gauss-Seidel è più caro degli altri, soprattutto sulle matrici **vem**: la sostituzione in avanti è sequenziale e non sfrutta la vettorizzazione come il prodotto matrice-vettore. Quindi Gauss-Seidel conviene quando il taglio nelle iterazioni ripaga il maggior costo del singolo passo, cosa che succede sulle **spa** ma non sulle **vem**.

4.9 Amplificazione errore/residuo

La Figura 7 mostra il rapporto tra errore relativo e residuo relativo, confrontato con il numero di condizionamento.

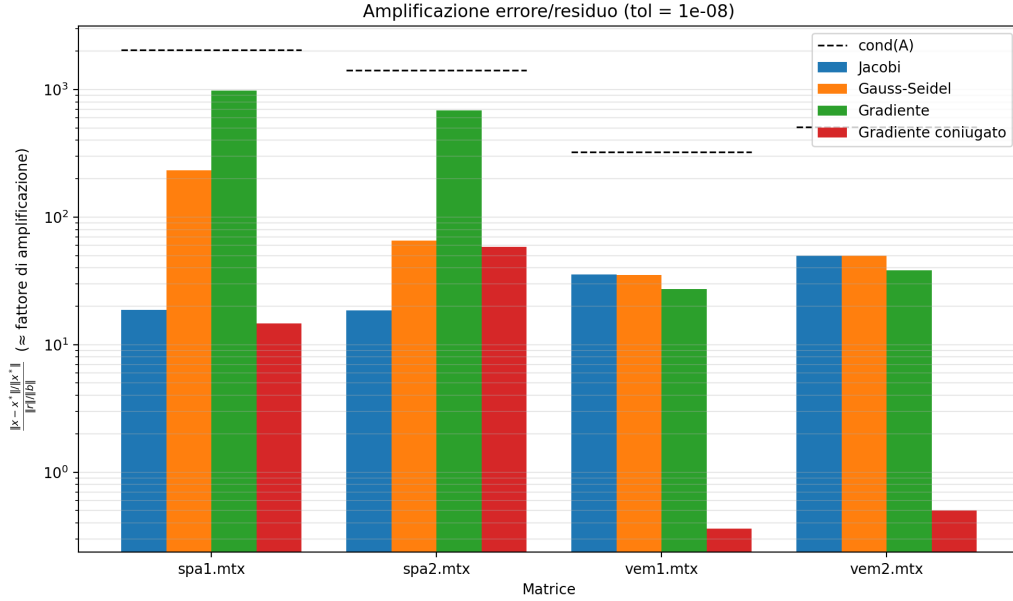


Figura 7: Rapporto fra errore relativo e residuo relativo a $\text{tol} = 10^{-8}$. La linea tratteggiata nera su ogni gruppo è $\text{cond}(A)$, limite superiore teorico del rapporto.

Commento. Il residuo piccolo non garantisce da solo un errore altrettanto piccolo: il legame tra i due passa per il condizionamento. Vale la stima errore relativo $\leq \kappa(A) \cdot \text{residuo relativo}$, e il grafico verifica che per ogni metodo il rapporto resta sotto la linea di $\text{cond}(A)$. Il limite è però pessimistico: i rapporti reali sono molto più bassi del condizionamento. Il gradiente semplice è quello che amplifica di più, vicino a 1000 sulle *spa*; il gradiente coniugato è quello che amplifica di meno e sulle matrici *vem* arriva sotto 1, cioè ottiene un errore addirittura più piccolo del proprio residuo. Questo spiega perché nel grafico dell'errore relativo il gradiente coniugato stava più in basso degli altri sulle *vem*.

4.10 Riepilogo

La Figura 8 raccoglie tempo e iterazioni in due grafici a barre, per un confronto diretto a $\text{tol} = 10^{-8}$.

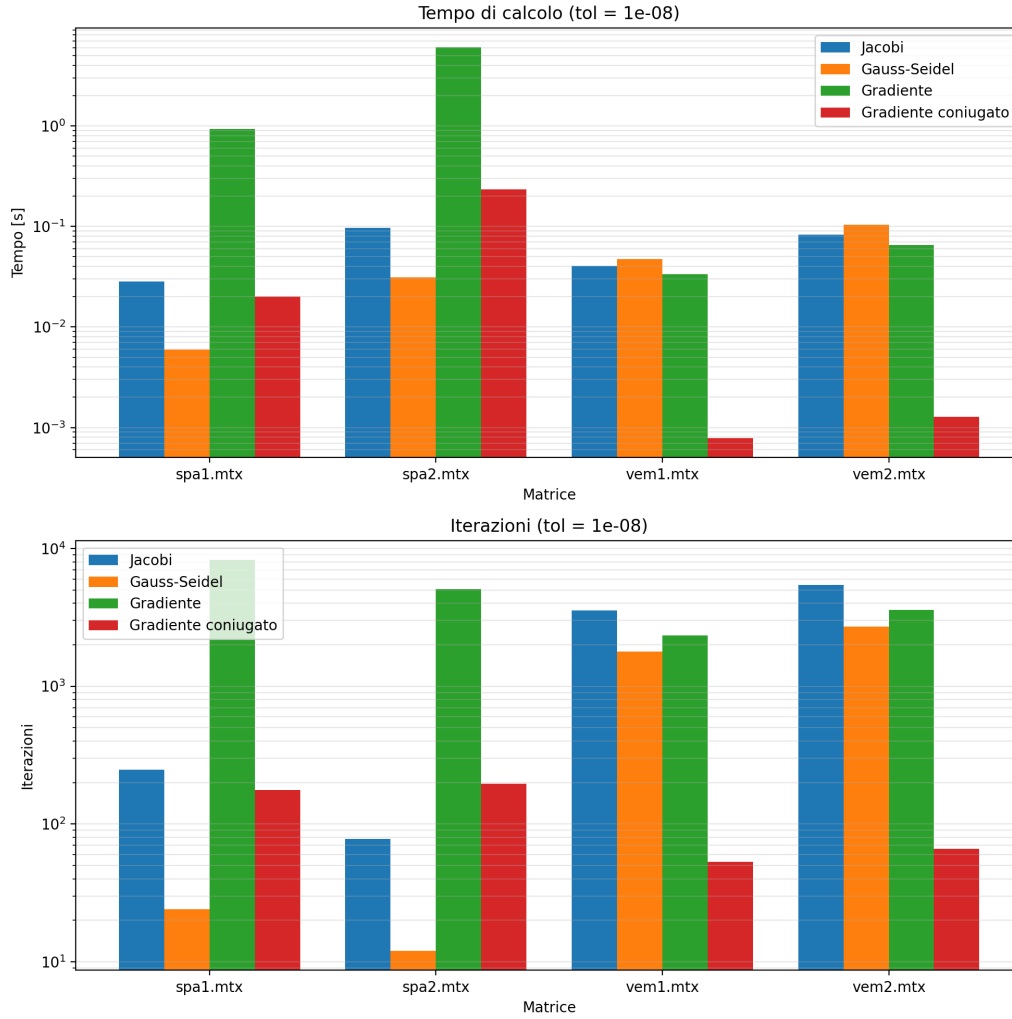


Figura 8: Confronto a barre del tempo (sopra) e delle iterazioni (sotto) a $\text{tol} = 10^{-8}$. Assi delle ordinate in scala logaritmica.

Commento. I due grafici riassumono tutto. In iterazioni vince sempre o quasi il gradiente coniugato, seguito da Gauss-Seidel; il gradiente semplice è il peggiore sulle *spa*. In tempo il quadro dipende dalla matrice: Gauss-Seidel è il più rapido sulle *spa* perché poche iterazioni ripagano il passo costoso, mentre il gradiente coniugato domina sulle *vem* grazie al numero piccolo di iterazioni. Il gradiente semplice resta la scelta peggiore quando la matrice è mal condizionata.

5 Conclusioni

Abbiamo implementato i quattro metodi richiesti in una libreria con uno scheletro comune e una regola di aggiornamento per metodo, e li abbiamo provati su quattro matrici reali a quattro tolleranze. I risultati sono coerenti con la teoria.

- Il criterio di arresto sul residuo scalato funziona: il residuo finale è sempre appena sotto la tolleranza e l'errore relativo scala con essa.
- Il gradiente coniugato è il metodo più robusto. Usa sempre pochissime iterazioni, in linea con la crescita $\sqrt{\kappa(A)}$, ed è il più veloce sulle matrici molto sparse.

- Gauss–Seidel converge in meno iterazioni di Jacobi ed è il più veloce sulle matrici di media densità, dove bastano pochi passi. Paga però un costo per iterazione più alto, perché la sostituzione in avanti è sequenziale.
- Il gradiente semplice è il più lento quando la matrice è mal condizionata, per via della convergenza a zig-zag. Il suo numero di iterazioni cresce come $\kappa(A)$.
- Il numero di condizionamento spiega bene il comportamento dei due metodi del gradiente. Per Jacobi e Gauss–Seidel conta invece anche la struttura della matrice, e l’andamento in funzione di $\kappa(A)$ non è monotono.

In sintesi, su questi problemi SPD il gradiente coniugato è la scelta di default più affidabile, mentre Gauss–Seidel può batterlo sul tempo quando la matrice è abbastanza densa da convergere in pochissime iterazioni.

6 Documentazione

Il codice è documentato in modo sistematico: ogni modulo ha una *docstring* di intestazione che ne descrive lo scopo, e ogni funzione o classe pubblica ha una *docstring* che ne specifica comportamento, argomenti, valore di ritorno ed eccezioni. Le firme usano le annotazioni di tipo (*type hints*). Di seguito riportiamo la documentazione per modulo. Per le funzioni di servizio interne, marcate con l’underscore iniziale, diamo solo una riga di descrizione.

6.1 Modulo `solvers.py`

È la libreria vera e propria. Contiene lo scheletro comune di iterazione, le quattro sottoclassi dei metodi e alcune funzioni di servizio.

`class IterativeSolver`. Classe base astratta con lo scheletro comune ai quattro metodi. Le sottoclassi forniscono solo la propria regola di aggiornamento, tramite il metodo astratto `_make_iteration`.

`IterativeSolver.solve(A, b, tol, max_iter=None, record_history=False)`. Risolve $A\mathbf{x} = \mathbf{b}$ partendo da $\mathbf{x}^{(0)} = \mathbf{0}$. Itera finché il residuo scalato scende sotto `tol` oppure finché si superano `max_iter` iterazioni; in questo secondo caso la soluzione è restituita con `converged=False`.

- *Input*: `A` (matrice sparsa CSR, $n \times n$), `b` (array, shape $(n,)$), `tol` (float), `max_iter` (int, default 20000), `record_history` (bool: se vero salva la storia del residuo per i grafici di convergenza).
- *Output*: `SolverResult`.
- *Eccezioni*: `ValueError` se `A` non è quadrata o ha un elemento diagonale nullo.

`class SolverResult`. *Dataclass* che raccoglie il risultato di una risoluzione: nome del metodo, soluzione `x`, numero di iterazioni, tempo `elapsed`, residuo scalato raggiunto, flag `converged` ed eventuale `history` del residuo.

`class Jacobi, GaussSeidel, Gradient, ConjugateGradient`. Le quattro sottoclassi concrete di `IterativeSolver`. Ognuna implementa solo la propria iterazione, cioè la regola di aggiornamento descritta nella Sezione 3, ed eredita il metodo `solve`.

`_gauss_seidel_sweep(indptr, indices, data, diag, b, x)`. Una iterazione (*sweep*) di Gauss–Seidel per sostituzione in avanti, che aggiorna `x` *in place*. È sequenziale e quindi compilata con `numba` per renderla veloce restando codice nostro. Lavora direttamente sugli array CSR della matrice.

`warmup()`. Forza la compilazione JIT di `numba` eseguendo i metodi su un problema 3×3 . Va chiamata prima delle misure di tempo, così la prima esecuzione di Gauss–Seidel non paga il costo di compilazione e i tempi non risultano falsati.

`SOLVERS`. Tupla con le istanze dei quattro metodi, nell'ordine richiesto dalla traccia.

6.2 Modulo `run_assignment.py`

È l'eseguibile da riga di comando che applica i quattro metodi alle matrici e salva i risultati.

`main()`. Legge le matrici, costruisce la soluzione esatta $\mathbf{x} = \mathbf{1}$ e il termine noto $\mathbf{b} = A\mathbf{x}$, applica i quattro metodi a ogni tolleranza, stampa a schermo errore relativo, iterazioni e tempo, scrive `results.csv` e, se richiesto, genera i grafici di base.

`parse_args()`. Legge gli argomenti da riga di comando.

- `-matrix`: matrice `.mtx` da usare, ripetibile; se assente usa tutti i file in `dati/`.
- `-tol`: tolleranza, ripetibile; default $10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}$.
- `-method`: metodo da usare fra `jacobi`, `gauss-seidel`, `gradient`, `conjugate-gradient`; ripetibile, default tutti.
- `-max-iter`: numero massimo di iterazioni, default 20000.
- `-output`: cartella di output, default `results`.
- `-no-plots`: non genera i grafici.

6.3 Modulo `plots.py`

Genera i grafici a partire dai file CSV. Riesegue i solutori solo per la storia di convergenza, che richiede di registrare il residuo a ogni iterazione.

`create_plots(rows, output_dir)`. Genera i tre grafici di base (errore, iterazioni e tempo in funzione della tolleranza). *Output*: lista dei percorsi dei file PNG scritti.

`compute_histories(tol, max_iter)`. Riesegue i solutori con `record_history=True` e restituisce, per ogni matrice e metodo, la storia del residuo scalato.

`plot_convergence`, `plot_iter_vs_cond`, `plot_error_vs_residual`, `plot_cost_per_iter`, `plot_summary_bars`. Le funzioni che disegnano i grafici di approfondimento della Sezione 4, uno per ciascuna figura.

6.4 Modulo `analyze_cond.py`

Script a margine usato solo per i commenti della relazione; non fa parte dei solutori.

`main()`. Per ogni matrice in `dati/` stima $\lambda_{\min}$ e $\lambda_{\max}$ con `scipy.sparse.linalg.eigsh`, calcola $\text{cond}(A) = \lambda_{\max}/\lambda_{\min}$ e scrive `cond.csv`.

6.5 Modulo `test_assignment.py`

Test rapidi su matrici piccole: controllano la lettura del formato MatrixMarket, la convergenza dei quattro metodi su una matrice tridiagonale SPD e il fatto che il gradiente coniugato converga in al più n iterazioni su un sistema piccolo.

6.6 Esecuzione

Dopo aver installato le dipendenze con `pip install -r requirements.txt`, l'esecuzione completa è

```
python run_assignment.py
```

che usa tutte le matrici in `dati/` e le quattro tolleranze richieste. Si possono scegliere matrice, tolleranza e metodo con gli argomenti descritti sopra:

```
python run_assignment.py --matrix dati/vem1.mtx --tol 1e-8
python run_assignment.py --method conjugate-gradient --no-plots
```

I grafici di approfondimento si generano con `python plots.py` dopo aver prodotto `results.csv` e `cond.csv`; i test con `python test_assignment.py`.

7 Codice

Per completezza riportiamo il codice sorgente dei file principali.

7.1 solvers.py

```
1 from __future__ import annotations
2
3 from abc import ABC, abstractmethod
4 from dataclasses import dataclass
5 from time import perf_counter
6 from typing import Any
7
8 import numpy as np
9 from numba import njit
10
11
12 # -----
13 # Risultato di una risoluzione.
14 # -----
15 @dataclass
16 class SolverResult:
17     method: str # nome del metodo
18     x: np.ndarray # soluzione approssimata  $x^{(k)}$ 
19     iterations: int # numero di iterazioni eseguite
20     elapsed: float # tempo di calcolo in secondi
21     residual_relative: float #  $\|b - Ax\|/\|b\|$  raggiunto
22     converged: bool # True se il criterio di arresto e' stato soddisfatto
23     history: list[float] | None = None # storia del residuo scalato (solo se richiesta)
24
25
26 # -----
27 # Iterazione astratta + classe base con lo scheletro comune.
28 # -----
29 class _Iteration(ABC):
30     """Stato interno di un metodo iterativo per un singolo problema.
31
32     Mantiene (e aggiorna in place) il vettore soluzione ' $x$ ' e quanto serve per
33     calcolare il residuo ed eseguire il passo di aggiornamento.
34     """
35
36     @abstractmethod
37     def residual_norm(self) -> float:
38         """Norma euclidea del residuo corrente  $\|b - Ax^{(k)}\|$ ."""
39
40     @abstractmethod
41     def step(self) -> None:
42         """Aggiorna lo stato:  $x^{(k)} \rightarrow x^{(k+1)}$ ."""
43
44
45 class IterativeSolver(ABC):
```

```

46     """Classe base: implementa lo scheletro comune a tutti i metodi."""
47
48     name: str = "iterative"
49     key: str = "iterative" # identificativo per la CLI
50     DEFAULT_MAX_ITER: int = 20000
51
52     @abstractmethod
53     def _make_iteration(self, A: Any, b: np.ndarray, x: np.ndarray) -> _Iteration:
54         """Costruisce l'oggetto iterazione specifico del metodo."""
55
56     def solve(self, A: Any, b: np.ndarray, tol: float,
57               max_iter: int | None = None,
58               record_history: bool = False) -> SolverResult:
59         """Risolve  $Ax = b$  partendo da  $x^{(0)} = 0$ .
60
61         Si arresta quando il residuo scalato scende sotto 'tol' oppure quando
62         si superano 'max_iter' iterazioni (in tal caso converged=False).
63
64         Se 'record_history=True' salva in 'SolverResult.history' il residuo
65         scalato a ogni iterazione (partendo dal valore iniziale = 1). Il valore
66         e' gia' calcolato dal ciclo, quindi non comporta matvec aggiuntivi; va
67         comunque usato SOLO per i grafici di convergenza, NON per le misure di
68         tempo (l'append alla lista introduce un piccolo overhead di Python).
69         """
70         _validate_for_iteration(A)
71         if max_iter is None:
72             max_iter = self.DEFAULT_MAX_ITER
73
74         b = np.asarray(b, dtype=float)
75         x = np.zeros_like(b, dtype=float) # vettore iniziale nullo
76         b_norm = _safe_norm(b)
77
78         start_time = perf_counter()
79         it = self._make_iteration(A, b, x)
80         iterations = 0
81         residual_relative = it.residual_norm() / b_norm # all'inizio vale 1
82         history = [residual_relative] if record_history else None
83         while residual_relative >= tol and iterations < max_iter:
84             it.step()
85             iterations += 1
86             residual_relative = it.residual_norm() / b_norm
87             if history is not None:
88                 history.append(residual_relative)
89         elapsed = perf_counter() - start_time
90
91         return SolverResult(
92             method=self.name,
93             x=x.copy(),
94             iterations=iterations,
95             elapsed=elapsed,
96             residual_relative=residual_relative,
97             converged=residual_relative < tol,
98             history=history,
99         )
100
101
102     # -----
103     # 1) Metodo di Jacobi.  $P = \text{diag}(A)$ ,  $x \leftarrow x + P^{-1} r$ ,  $r = b - Ax$ .
104     # -----
105     class _JacobiIteration(_Iteration):
106         def __init__(self, A, b, x):
107             self.A = A
108             self.b = b

```

```

109     self.x = x
110     self.inv_diag = 1.0 / np.asarray(A.diagonal(), dtype=float) #  $P^{-1}$ 
111     self.residual = b - A @ x # residuo iniziale (= b)
112
113     def residual_norm(self):
114         return float(np.linalg.norm(self.residual))
115
116     def step(self):
117         #  $x^{(k+1)} = x^{(k)} + P^{-1} r^{(k)}$ 
118         self.x += self.inv_diag * self.residual
119         self.residual = self.b - self.A @ self.x
120
121
122 class Jacobi(IterativeSolver):
123     name = "Jacobi"
124     key = "jacobi"
125
126     def _make_iteration(self, A, b, x):
127         return _JacobiIteration(A, b, x)
128
129
130 # -----
131 # 2) Metodo di Gauss-Seidel.  $P$  = parte triangolare inferiore di  $A$ .
132 # sweep in place (= sostituzione in avanti), poi  $r = b - A x$ .
133 # -----
134 class _GaussSeidelIteration(_Iteration):
135     def __init__(self, A, b, x):
136         self.A = A
137         self.b = b
138         self.x = x
139         self.diag = np.asarray(A.diagonal(), dtype=float)
140         self.indptr = A.indptr
141         self.indices = A.indices
142         self.data = np.asarray(A.data, dtype=float)
143         self.residual = b - A @ x
144
145     def residual_norm(self):
146         return float(np.linalg.norm(self.residual))
147
148     def step(self):
149         _gauss_seidel_sweep(self.indptr, self.indices, self.data,
150                             self.diag, self.b, self.x)
151         self.residual = self.b - self.A @ self.x
152
153
154 class GaussSeidel(IterativeSolver):
155     name = "Gauss-Seidel"
156     key = "gauss-seidel"
157
158     def _make_iteration(self, A, b, x):
159         return _GaussSeidelIteration(A, b, x)
160
161
162 # -----
163 # 3) Metodo del Gradiente (steepest descent).
164 #  $\alpha = (r \cdot r) / (r \cdot A r)$  ;  $x \leftarrow x + \alpha r$  ;  $r \leftarrow r - \alpha A r$ .
165 # -----
166 class _GradientIteration(_Iteration):
167     def __init__(self, A, b, x):
168         self.A = A
169         self.b = b
170         self.x = x
171         self.residual = b - A @ x # = -grad(phi) in x

```

```

172
173 def residual_norm(self):
174     return float(np.linalg.norm(self.residual))
175
176 def step(self):
177     Ar = self.A @ self.residual
178     denom = float(np.dot(self.residual, Ar))
179     alpha = float(np.dot(self.residual, self.residual)) / denom
180     self.x += alpha * self.residual
181     #  $r^{(k+1)} = b - A x^{(k+1)} = r^{(k)} - \alpha A r^{(k)}$ : niente matvec extra
182     self.residual = self.residual - alpha * Ar
183
184
185 class Gradient(IterativeSolver):
186     name = "Gradiente"
187     key = "gradient"
188
189     def _make_iteration(self, A, b, x):
190         return _GradientIteration(A, b, x)
191
192
193 # -----
194 # 4) Metodo del Gradiente coniugato.
195 # Direzioni A-coniugate: niente "zig-zag", convergenza in  $\leq n$  passi.
196 # -----
197 class _ConjugateGradientIteration(_Iteration):
198     def __init__(self, A, b, x):
199         self.A = A
200         self.b = b
201         self.x = x
202         self.residual = b - A @ x # residuo iniziale (= b)
203         self.direction = self.residual.copy() #  $d^{(0)} = r^{(0)}$ 
204         self.residual_norm_sq = float(np.dot(self.residual, self.residual))
205
206     def residual_norm(self):
207         return float(np.sqrt(self.residual_norm_sq))
208
209     def step(self):
210         Ad = self.A @ self.direction
211         denom = float(np.dot(self.direction, Ad))
212         alpha = self.residual_norm_sq / denom
213         self.x += alpha * self.direction
214         self.residual = self.residual - alpha * Ad
215         new_norm_sq = float(np.dot(self.residual, self.residual))
216         beta = new_norm_sq / self.residual_norm_sq
217         self.direction = self.residual + beta * self.direction
218         self.residual_norm_sq = new_norm_sq
219
220
221 class ConjugateGradient(IterativeSolver):
222     name = "Gradiente coniugato"
223     key = "conjugate-gradient"
224
225     def _make_iteration(self, A, b, x):
226         return _ConjugateGradientIteration(A, b, x)
227
228
229 # Istanze dei quattro metodi, nell'ordine richiesto dalla consegna.
230 SOLVERS: tuple[IterativeSolver, ...] = (
231     Jacobi(),
232     GaussSeidel(),
233     Gradient(),
234     ConjugateGradient(),

```



```

235 )
236
237
238 # -----
239 # Routine compilata (numba) e funzioni di servizio condivise.
240 # -----
241 @njit(cache=True)
242 def _gauss_seidel_sweep(indptr, indices, data, diag, b, x):
243     """Una iterazione (sweep) di Gauss-Seidel, aggiornando x in place.
244
245     Equivale a risolvere  $(D + L)y = r$  per sostituzione in avanti: ogni  $x[i]$  usa
246     i valori già aggiornati  $x[0..i-1]$  e quelli vecchi  $x[i+1..n-1]$ . E'
247     un'operazione sequenziale ( $x[i]$  dipende dai precedenti) e quindi non
248     vettorizzabile: la compiliamo con numba per renderla veloce restando codice
249     nostro (non usiamo nessun solutore di libreria).
250     """
251     n = x.shape[0]
252     for i in range(n):
253         sigma = 0.0
254         for k in range(indptr[i], indptr[i + 1]):
255             j = indices[k]
256             if j != i:
257                 sigma += data[k] * x[j]
258             x[i] = (b[i] - sigma) / diag[i]
259
260
261 def warmup() -> None:
262     """Forza la compilazione JIT (numba) su un problemino 3x3.
263
264     Va chiamata PRIMA delle misure di tempo: altrimenti la prima esecuzione di
265     Gauss-Seidel paga il costo (una tantum) di compilazione della sweep,
266     falsando il tempo riportato.
267     """
268     from scipy.sparse import csr_matrix
269
270     A = csr_matrix(np.array([[4.0, 1.0, 0.0],
271                             [1.0, 4.0, 1.0],
272                             [0.0, 1.0, 4.0]]))
273     b = A @ np.ones(3)
274     for solver in SOLVERS:
275         solver.solve(A, b, 1e-12, 100)
276
277
278 def _validate_for_iteration(A: Any) -> None:
279     if A.shape[0] != A.shape[1]:
280         raise ValueError("La matrice deve essere quadrata")
281     diag = np.asarray(A.diagonal(), dtype=float)
282     if np.any(diag == 0):
283         raise ValueError("La matrice ha almeno un elemento diagonale nullo")
284
285
286 def _safe_norm(x: np.ndarray) -> float:
287     norm = float(np.linalg.norm(x))
288     if norm == 0:
289         return 1.0
290     return norm

```

7.2 run_assignment.py

```

1  """Command line runner for the first MCS assignment."""
2

```

```

3 from __future__ import annotations
4
5 import argparse
6 import csv
7 from pathlib import Path
8
9 import numpy as np
10 from scipy.io import mmread
11
12 from plots import create_plots
13 from solvers import SOLVERS, warmup
14
15
16 DEFAULT_TOLS = (1e-4, 1e-6, 1e-8, 1e-10)
17 DEFAULT_MAX_ITER = 20000
18 METHOD_NAMES = tuple(solver.key for solver in SOLVERS)
19
20
21 def main() -> None:
22     args = parse_args()
23     base_dir = Path(__file__).resolve().parent
24     output_dir = args.output
25     if not output_dir.is_absolute():
26         output_dir = base_dir / output_dir
27     output_dir.mkdir(parents=True, exist_ok=True)
28
29     matrix_paths = _resolve_matrices(args, base_dir)
30     tolerances = args.tol if args.tol else list(DEFAULT_TOLS)
31     solvers = _select_solvers(args.method)
32
33     warmup() # compilazione JIT (numba) prima delle misure di tempo
34
35     rows: list[dict[str, str]] = []
36     for matrix_path in matrix_paths:
37         print(f"\nMatrice: {matrix_path}")
38         A = mmread(matrix_path).tocsr().astype(float)
39         x_exact = np.ones(A.shape[0], dtype=float)
40         b = A @ x_exact
41
42         for tol in tolerances:
43             print(f" tol = {tol:g}")
44             for solver in solvers:
45                 result = solver.solve(A, b, tol, args.max_iter)
46                 relative_error = np.linalg.norm(x_exact - result.x) / np.linalg.norm(x_exact)
47                 row = {
48                     "matrix": matrix_path.name,
49                     "n": str(A.shape[0]),
50                     "nnz": str(A.nnz),
51                     "tol": f"{tol:.0e}",
52                     "method": result.method,
53                     "relative_error": f"{relative_error:.16e}",
54                     "residual_relative": f"{result.residual_relative:.16e}",
55                     "iterations": str(result.iterations),
56                     "elapsed_seconds": f"{result.elapsed:.6f}",
57                     "converged": str(result.converged),
58                 }
59                 rows.append(row)
60             print(
61                 " "
62                 f"{result.method:20s} "
63                 f"it={result.iterations:6d} "
64                 f"err={relative_error:.3e} "
65                 f"res={result.residual_relative:.3e} "

```

```

66         f"time={result.elapsed:.3f}s "
67         f"conv={result.converged}"
68     )
69
70     csv_path = output_dir / "results.csv"
71     _write_csv(rows, csv_path)
72     print(f"\nCSV scritto in: {csv_path}")
73
74     if args.plots:
75         plot_paths = create_plots(rows, output_dir)
76         print(f"Grafici scritti in: {output_dir / 'plots'}")
77         for plot_path in plot_paths:
78             print(f" - {plot_path.name}")
79
80
81 def parse_args() -> argparse.Namespace:
82     parser = argparse.ArgumentParser(description="Mini libreria per sistemi lineari SPD")
83     parser.add_argument(
84         "--matrix",
85         action="append",
86         type=Path,
87         help="Matrice .mtx da usare. Ripetibile. Se assente usa tutti i file .mtx in dati.",
88     )
89     parser.add_argument(
90         "--tol",
91         action="append",
92         type=float,
93         help="Tolleranza da usare. Ripetibile. Se assente usa 1e-4, 1e-6, 1e-8, 1e-10.",
94     )
95     parser.add_argument(
96         "--max-iter",
97         type=int,
98         default=DEFAULT_MAX_ITER,
99         help="Numero massimo di iterazioni, default 20000.",
100     )
101     parser.add_argument(
102         "--method",
103         action="append",
104         choices=METHOD_NAMES,
105         help=(
106             "Metodo da usare. Ripetibile. Se assente usa tutti i metodi. "
107             f"Valori: {' ', ' '.join(METHOD_NAMES)}."
108         ),
109     )
110     parser.add_argument(
111         "--output",
112         type=Path,
113         default=Path("results"),
114         help="Cartella di output, default assignment 1/results.",
115     )
116     parser.add_argument(
117         "--no-plots",
118         dest="plots",
119         action="store_false",
120         help="Non generare grafici.",
121     )
122     parser.set_defaults(plots=True)
123     return parser.parse_args()
124
125
126 def _resolve_matrices(args: argparse.Namespace, base_dir: Path) -> list[Path]:
127     if args.matrix:
128         paths = args.matrix

```

```

129     else:
130         data_dir = base_dir / "dati"
131         paths = sorted(data_dir.glob("*.mtx"))
132         if not paths:
133             raise FileNotFoundError(f"Nessuna matrice .mtx trovata in: {data_dir}")
134
135     resolved = []
136     for path in paths:
137         if not path.is_absolute():
138             path = base_dir / path
139         if not path.exists():
140             raise FileNotFoundError(f"Matrice non trovata: {path}")
141         resolved.append(path)
142     return resolved
143
144
145 def _select_solvers(methods: list[str] | None):
146     if not methods:
147         return SOLVERS
148
149     selected = set(methods)
150     return tuple(solver for solver in SOLVERS if solver.key in selected)
151
152
153 def _write_csv(rows: list[dict[str, str]], csv_path: Path) -> None:
154     fieldnames = [
155         "matrix",
156         "n",
157         "nnz",
158         "tol",
159         "method",
160         "relative_error",
161         "residual_relative",
162         "iterations",
163         "elapsed_seconds",
164         "converged",
165     ]
166     with csv_path.open("w", encoding="utf-8", newline="") as file:
167         writer = csv.DictWriter(file, fieldnames=fieldnames)
168         writer.writeheader()
169         writer.writerows(rows)
170
171
172 if __name__ == "__main__":
173     main()

```

7.3 analyze_cond.py

```

1  """Stima del numero di condizionamento delle matrici di test.
2
3  NOTA: questo script serve SOLO all'analisi dei risultati nella relazione
4  (spiegare perche' alcuni metodi sono piu' lenti). NON fa parte della mini
5  libreria di solutori: qui usiamo scipy.sparse.linalg.eigsh, che e' un
6  calcolatore di autovalori, per stimare lambda_max e lambda_min e quindi
7  cond(A) = lambda_max / lambda_min.
8
9  Uso:
10     python analyze_cond.py
11 """
12 from __future__ import annotations
13

```

```

14 import csv
15 from pathlib import Path
16
17 from scipy.io import mmread
18 from scipy.sparse.linalg import eigsh
19
20 BASE_DIR = Path(__file__).resolve().parent
21 DATA_DIR = BASE_DIR / "dati"
22 OUTPUT_DIR = BASE_DIR / "results"
23
24
25 def main() -> None:
26     paths = sorted(DATA_DIR.glob("*.mtx"))
27     print(f'{"matrice":<12s}{"n":>7s}{"lambda_min":>14s}{"lambda_max":>14s}{"cond(A)":>14s}'
28         )
29     print("-" * 61)
30     rows: list[dict[str, object]] = []
31     for path in paths:
32         A = mmread(path).tocsr().astype(float)
33         n = A.shape[0]
34         lam_max = float(eigsh(A, k=1, which="LM", return_eigenvectors=False)[0])
35         # piu' piccolo autovalore via shift-invert attorno a 0 (A SPD => lambda_min > 0)
36         lam_min = float(eigsh(A, k=1, sigma=0.0, which="LM",
37                             return_eigenvectors=False)[0])
38         cond = lam_max / lam_min
39         print(f'{"path.name":<12s}{"n":>7d}{"lam_min":>14.3e}{"lam_max":>14.3e}{"cond":>14.3e}')
40         rows.append({
41             "matrix": path.name,
42             "n": n,
43             "lambda_min": f"{lam_min:.16e}",
44             "lambda_max": f"{lam_max:.16e}",
45             "cond": f"{cond:.16e}",
46         })
47
48     # Salviamo i valori anche su CSV: servono ai grafici (plots.py) e ad
49     # arricchire le tabelle della relazione.
50     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
51     cond_csv = OUTPUT_DIR / "cond.csv"
52     with cond_csv.open("w", encoding="utf-8", newline="") as file:
53         writer = csv.DictWriter(
54             file, fieldnames=["matrix", "n", "lambda_min", "lambda_max", "cond"])
55         writer.writeheader()
56         writer.writerows(rows)
57     print(f"\nCSV scritto in: {cond_csv}")
58
59 if __name__ == "__main__":
60     main()

```

7.4 plots.py

```

1 from __future__ import annotations
2
3 import argparse
4 import csv
5 from collections import defaultdict
6 from pathlib import Path
7
8 import numpy as np
9 from scipy.io import mmread
10 from scipy.sparse import csr_matrix

```

```

11
12 from solvers import SOLVERS, warmup
13
14 BASE_DIR = Path(__file__).resolve().parent
15 DATA_DIR = BASE_DIR / "dati"
16
17 # Ordine e colori dei metodi, coerenti in tutti i grafici di approfondimento.
18 METHOD_ORDER = [solver.name for solver in SOLVERS]
19 METHOD_COLORS = {
20     "Jacobi": "tab:blue",
21     "Gauss-Seidel": "tab:orange",
22     "Gradiente": "tab:green",
23     "Gradiente coniugato": "tab:red",
24 }
25
26
27 # -----
28 # Funzioni di servizio condivise.
29 # -----
30 def _save(fig, path: Path) -> None:
31     """Layout, salvataggio a 200 dpi e chiusura della figura."""
32     import matplotlib.pyplot as plt
33
34     fig.tight_layout()
35     fig.savefig(path, dpi=200)
36     plt.close(fig)
37
38
39 def load_rows(csv_path: Path) -> list[dict[str, str]]:
40     if not csv_path.exists():
41         raise FileNotFoundError(
42             f"Risultati non trovati: {csv_path}. Esegui prima 'python run_assignment.py'."
43         )
44     with csv_path.open(encoding="utf-8", newline="") as file:
45         return list(csv.DictReader(file))
46
47
48 def load_cond(cond_csv: Path) -> dict[str, float]:
49     if not cond_csv.exists():
50         raise FileNotFoundError(
51             f"File dei condizionamenti non trovato: {cond_csv}. "
52             f"Esegui prima 'python analyze_cond.py'."
53         )
54     with cond_csv.open(encoding="utf-8", newline="") as file:
55         return {row["matrix"]: float(row["cond"]) for row in csv.DictReader(file)}
56
57
58 def _closest_tol(rows: list[dict[str, str]], target: float) -> str:
59     """Etichetta di tolleranza presente nel CSV piu' vicina a 'target'."""
60     tols = sorted({row["tol"] for row in rows}, key=float)
61     return min(tols, key=lambda t: abs(np.log10(float(t)) - np.log10(target)))
62
63
64 def _methods_present(rows: list[dict[str, str]]) -> list[str]:
65     present = {row["method"] for row in rows}
66     return [m for m in METHOD_ORDER if m in present]
67
68
69 def _grouped_bars(ax, groups, series, value_fn) -> None:
70     """Disegna barre raggruppate: un gruppo per 'groups', una barra per
71     'series'. 'value_fn(group, serie)' ritorna l'altezza (o None)."""
72     n_series = max(len(series), 1)
73     width = 0.8 / n_series
74     x = np.arange(len(groups))
75     for j, serie in enumerate(series):

```

```

74     offset = (j - (n_series - 1) / 2) * width
75     heights = [value_fn(group, serie) or np.nan for group in groups]
76     ax.bar(x + offset, heights, width=width, label=serie,
77           color=METHOD_COLORS.get(serie))
78
79
80 # -----
81 # Grafici fondamentali: metrica vs tolleranza (richiamati da run_assignment.py).
82 # -----
83 def create_plots(rows: list[dict[str, str]], output_dir: str | Path) -> list[Path]:
84     output_dir = Path(output_dir)
85     plots_dir = output_dir / "plots"
86     plots_dir.mkdir(parents=True, exist_ok=True)
87
88     paths = [
89         plots_dir / "relative_error.png",
90         plots_dir / "iterations.png",
91         plots_dir / "time.png",
92     ]
93     _plot_metric(rows, "relative_error", "Errore relativo", paths[0], log_y=True)
94     _plot_metric(rows, "iterations", "Iterazioni", paths[1], log_y=False)
95     _plot_metric(rows, "elapsed_seconds", "Tempo [s]", paths[2], log_y=True)
96     return paths
97
98
99 def _plot_metric(
100     rows: list[dict[str, str]],
101     metric: str,
102     ylabel: str,
103     path: Path,
104     log_y: bool,
105 ) -> None:
106     import matplotlib.pyplot as plt
107
108     grouped = defaultdict(list)
109     for row in rows:
110         grouped[(row["matrix"], row["method"])].append(row)
111
112     matrices = sorted({row["matrix"] for row in rows})
113     fig, axes = plt.subplots(len(matrices), 1, figsize=(9, 3.2 * len(matrices)), squeeze=
114                             False)
115
116     for ax, matrix_name in zip(axes[:, 0], matrices):
117         for (current_matrix, method), values in grouped.items():
118             if current_matrix != matrix_name:
119                 continue
120             values = sorted(values, key=lambda item: float(item["tol"]))
121             xs = [float(item["tol"]) for item in values]
122             ys = [float(item[metric]) for item in values]
123             ax.plot(xs, ys, marker="o", label=method)
124
125             ax.set_title(matrix_name)
126             ax.set_xlabel("Tolleranza")
127             ax.set_ylabel(ylabel)
128             ax.set_xscale("log")
129             ax.invert_xaxis()
130             if log_y:
131                 ax.set_yscale("log")
132             ax.grid(True, which="both", alpha=0.3)
133             ax.legend()
134
135     _save(fig, path)

```

```

136
137 # -----
138 # A) Storia di convergenza.
139 # -----
140 def compute_histories(tol: float, max_iter: int) -> dict[str, dict[str, list[float]]]:
141     """Riesegue i solutori registrando la storia del residuo scalato.
142
143     Ritorna histories[matrice][metodo] = [r0, r1, ...] (r0 = 1).
144     """
145     warmup()
146     paths = sorted(DATA_DIR.glob("*.mtx"))
147     histories: dict[str, dict[str, list[float]]] = {}
148     for path in paths:
149         A = csr_matrix(mread(path), dtype=float)
150         # .shape e' tipato Optional negli stub di scipy: a runtime e' sempre una
151         # tupla (n, n), quindi il pedice e' sicuro.
152         x_exact = np.ones(A.shape[0], dtype=float) # type: ignore[reportOptionalSubscript]
153         b = A @ x_exact
154         per_method: dict[str, list[float]] = {}
155         for solver in SOLVERS:
156             result = solver.solve(A, b, tol, max_iter, record_history=True)
157             per_method[solver.name] = result.history or []
158             histories[path.name] = per_method
159             print(f" storia calcolata: {path.name}")
160     return histories
161
162
163 def plot_convergence(histories: dict[str, dict[str, list[float]]], tol: float,
164                     path: Path) -> None:
165     import matplotlib.pyplot as plt
166
167     matrices = sorted(histories)
168     fig, axes = plt.subplots(len(matrices), 1, figsize=(9, 3.2 * len(matrices)),
169                             squeeze=False)
170     for ax, matrix_name in zip(axes[:, 0], matrices):
171         for method in METHOD_ORDER:
172             history = histories[matrix_name].get(method)
173             if not history or len(history) < 2:
174                 continue
175             # Saltiamo il punto iniziale (r0 = 1, k = 0) perche' l'asse x e' in
176             # scala logaritmica: cosi' tutti i metodi sono visibili anche quando
177             # il numero di iterazioni varia di ordini di grandezza (es. CG ~10^2
178             # vs Gradiente ~10^4 sulle matrici spa).
179             tail = history[1:]
180             ax.plot(range(1, len(tail) + 1), tail, label=method,
181                     color=METHOD_COLORS.get(method))
182             ax.axhline(tol, color="gray", linestyle="--", linewidth=1,
183                       label=f"tol = {tol:g}")
184             ax.set_title(matrix_name)
185             ax.set_xlabel("Iterazione k")
186             ax.set_ylabel(r"$\|r_k\| / \|b\|$")
187             ax.set_xscale("log")
188             ax.set_yscale("log")
189             ax.grid(True, which="both", alpha=0.3)
190             ax.legend()
191         _save(fig, path)
192
193
194 # -----
195 # B) Iterazioni vs cond(A).
196 # -----
197 def plot_iter_vs_cond(rows: list[dict[str, str]], cond_map: dict[str, float],
198                      tol_label: str, path: Path) -> None:

```



```

199 import matplotlib.pyplot as plt
200
201 selected = [row for row in rows if row["tol"] == tol_label]
202 fig, ax = plt.subplots(figsize=(9, 6))
203
204 for method in _methods_present(selected):
205     points = [(cond_map[row["matrix"]], int(row["iterations"]))
206               for row in selected
207               if row["method"] == method and row["matrix"] in cond_map]
208     points.sort()
209     if not points:
210         continue
211     conds = [p[0] for p in points]
212     iters = [p[1] for p in points]
213     ax.plot(conds, iters, marker="o", label=method,
214            color=METHOD_COLORS.get(method))
215
216     # Curve teoriche di riferimento, ancorate al punto a cond minimo del
217     # Gradiente / CG (numero di iterazioni ~ cond per il Gradiente, ~sqrt(cond)
218     # per il CG).
219     conds_axis = np.array(sorted(cond_map.values()))
220     _add_reference_curve(ax, selected, cond_map, "Gradiente", conds_axis,
221                        lambda c, c0: c / c0, r"$\propto \kappa(A)$")
222     _add_reference_curve(ax, selected, cond_map, "Gradiente coniugato",
223                        conds_axis, lambda c, c0: np.sqrt(c / c0),
224                        r"$\propto \sqrt{\kappa(A)}$")
225
226     ax.set_title(f"Iterazioni vs numero di condizionamento (tol = {tol_label})")
227     ax.set_xlabel(r"$\kappa(A) = \lambda_{\max} / \lambda_{\min}$")
228     ax.set_ylabel("Iterazioni")
229     ax.set_xscale("log")
230     ax.set_yscale("log")
231     ax.grid(True, which="both", alpha=0.3)
232     ax.legend()
233     _save(fig, path)
234
235
236 def _add_reference_curve(ax, selected, cond_map, method, conds_axis,
237                        shape, label) -> None:
238     """Disegna una curva teorica ancorata al dato reale a cond minimo."""
239     points = [(cond_map[row["matrix"]], int(row["iterations"]))
240               for row in selected
241               if row["method"] == method and row["matrix"] in cond_map]
242     if not points:
243         return
244     points.sort()
245     c0, it0 = points[0]
246     ys = [it0 * shape(c, c0) for c in conds_axis]
247     ax.plot(conds_axis, ys, linestyle=":", color=METHOD_COLORS.get(method),
248            alpha=0.7, label=f"{method}: {label}")
249
250
251 # -----
252 # C) Amplificazione errore/residuo.
253 # -----
254 def plot_error_vs_residual(rows: list[dict[str, str]], cond_map: dict[str, float],
255                           tol_label: str, path: Path) -> None:
256     import matplotlib.pyplot as plt
257
258     selected = [row for row in rows if row["tol"] == tol_label]
259     matrices = sorted({row["matrix"] for row in selected})
260     methods = _methods_present(selected)
261

```

```

262 ratios = defaultdict(dict) # ratios[matrix][method] = err/res
263 for row in selected:
264     res = float(row["residual_relative"])
265     if res > 0:
266         ratios[row["matrix"]][row["method"]] = float(row["relative_error"]) / res
267
268 fig, ax = plt.subplots(figsize=(10, 6))
269 _grouped_bars(ax, matrices, methods,
270               lambda mat, met: ratios.get(mat, {}).get(met))
271
272 # Riferimento: il rapporto err/res e' limitato superiormente da cond(A).
273 for i, matrix_name in enumerate(matrices):
274     if matrix_name in cond_map:
275         ax.hlines(cond_map[matrix_name], i - 0.4, i + 0.4,
276                  color="black", linestyle="--", linewidth=1.2,
277                  label="cond(A)" if i == 0 else "_nolegend_")
278
279 ax.set_title(f"Amplificazione errore/residuo (tol = {tol_label})")
280 ax.set_xlabel("Matrice")
281 ax.set_ylabel(r"$\frac{\|x-x^*\|}{\|x^*\|}\{\|r\|/\|b\|\}$ ($\approx$ fattore di amplificazione)")
282 ax.set_yscale("log")
283 ax.set_xticks(range(len(matrices)))
284 ax.set_xticklabels(matrices)
285 ax.grid(True, axis="y", which="both", alpha=0.3)
286 ax.legend()
287 _save(fig, path)
288
289 # -----
290 # D) Costo per iterazione + bar chart riassuntivi.
291 # -----
292
293 def plot_cost_per_iter(rows: list[dict[str, str]], tol_label: str, path: Path) -> None:
294     import matplotlib.pyplot as plt
295
296     selected = [row for row in rows if row["tol"] == tol_label]
297     matrices = sorted({row["matrix"] for row in selected})
298     methods = _methods_present(selected)
299
300     cost = defaultdict(dict) # cost[matrix][method] = elapsed/iterations
301     for row in selected:
302         iters = int(row["iterations"])
303         if iters > 0:
304             cost[row["matrix"]][row["method"]] = float(row["elapsed_seconds"]) / iters
305
306     fig, ax = plt.subplots(figsize=(10, 6))
307     _grouped_bars(ax, matrices, methods,
308                   lambda mat, met: cost.get(mat, {}).get(met))
309     ax.set_title(f"Costo medio per iterazione (tol = {tol_label})")
310     ax.set_xlabel("Matrice")
311     ax.set_ylabel("Tempo per iterazione [s]")
312     ax.set_yscale("log")
313     ax.set_xticks(range(len(matrices)))
314     ax.set_xticklabels(matrices)
315     ax.grid(True, axis="y", which="both", alpha=0.3)
316     ax.legend()
317     _save(fig, path)
318
319
320 def plot_summary_bars(rows: list[dict[str, str]], tol_label: str, path: Path) -> None:
321     import matplotlib.pyplot as plt
322
323     selected = [row for row in rows if row["tol"] == tol_label]
324     matrices = sorted({row["matrix"] for row in selected})

```

```

325 methods = _methods_present(selected)
326
327 time_of = defaultdict(dict)
328 iter_of = defaultdict(dict)
329 for row in selected:
330     time_of[row["matrix"]][row["method"]] = float(row["elapsed_seconds"])
331     iter_of[row["matrix"]][row["method"]] = int(row["iterations"])
332
333 fig, axes = plt.subplots(2, 1, figsize=(10, 10))
334 _grouped_bars(axes[0], matrices, methods,
335               lambda mat, met: time_of.get(mat, {}).get(met))
336 axes[0].set_title(f"Tempo di calcolo (tol = {tol_label})")
337 axes[0].set_ylabel("Tempo [s]")
338 axes[0].set_yscale("log")
339
340 _grouped_bars(axes[1], matrices, methods,
341               lambda mat, met: iter_of.get(mat, {}).get(met))
342 axes[1].set_title(f"Iterazioni (tol = {tol_label})")
343 axes[1].set_ylabel("Iterazioni")
344 axes[1].set_yscale("log")
345
346 for ax in axes:
347     ax.set_xlabel("Matrice")
348     ax.set_xticks(range(len(matrices)))
349     ax.set_xticklabels(matrices)
350     ax.grid(True, axis="y", which="both", alpha=0.3)
351     ax.legend()
352 _save(fig, path)
353
354
355 # -----
356 # Main: genera i grafici di approfondimento dai CSV (e dalla riesecuzione dei
357 # solutori per la storia di convergenza).
358 # -----
359 def main() -> None:
360     args = parse_args()
361     output_dir = args.output
362     if not output_dir.is_absolute():
363         output_dir = BASE_DIR / output_dir
364     plots_dir = output_dir / "plots"
365     plots_dir.mkdir(parents=True, exist_ok=True)
366
367     rows = load_rows(output_dir / "results.csv")
368     cond_map = load_cond(output_dir / "cond.csv")
369
370     bar_tol = _closest_tol(rows, args.bar_tol)
371
372     print("Genero i grafici dal CSV...")
373     plot_iter_vs_cond(rows, cond_map, bar_tol, plots_dir / "iter_vs_cond.png")
374     plot_error_vs_residual(rows, cond_map, bar_tol, plots_dir / "error_vs_residual.png")
375     plot_cost_per_iter(rows, bar_tol, plots_dir / "cost_per_iter.png")
376     plot_summary_bars(rows, bar_tol, plots_dir / "summary_bars.png")
377
378     print("Calcolo la storia di convergenza (rieseguo i solutori)...")
379     histories = compute_histories(args.conv_tol, args.max_iter)
380     plot_convergence(histories, args.conv_tol, plots_dir / "convergence.png")
381
382     print(f"\nGrafici scritti in: {plots_dir}")
383     for name in ("convergence.png", "iter_vs_cond.png", "error_vs_residual.png",
384                "cost_per_iter.png", "summary_bars.png"):
385         print(f" - {name}")
386
387

```

```

388 def parse_args() -> argparse.Namespace:
389     parser = argparse.ArgumentParser(
390         description="Grafici di approfondimento (MCS Assignment 1)")
391     parser.add_argument("--output", type=Path, default=Path("results"),
392         help="Cartella con results.csv/cond.csv e output plots/, default
           results.")
393     parser.add_argument("--conv-tol", type=float, default=1e-10,
394         help="Tolleranza per la storia di convergenza, default 1e-10.")
395     parser.add_argument("--bar-tol", type=float, default=1e-8,
396         help="Tolleranza per i grafici a barre / cond, default 1e-8.")
397     parser.add_argument("--max-iter", type=int, default=20000,
398         help="Numero massimo di iterazioni, default 20000.")
399     return parser.parse_args()
400
401
402 if __name__ == "__main__":
403     main()

```